

UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ESTRUCTURA DE DATOS



Mauricio Joel Gómez Barrios - 202031478

26 de febrero de 2026

Índice.

Introducción.....	3
Organización del Proyecto y Diagrama de Clases.....	4
Análisis de gramática: Analizador léxico (Lexer.flex).....	4
• Configuraciones y declaraciones generales.....	4
• Expresiones Regulares.....	4
• Clasificación de tokens (Reglas léxicas).....	5
• Manejo y Recuperación de Errores Léxicos.....	5
Análisis de Gramática: Analizador Sintáctico (Parser.cup).....	6
Código de usuario y estructuras de datos.....	6
Precedencia y asociatividad de operadores.....	6
Reglas de Producción.....	6
Recuperación de Errores.....	7
Controlador Principal (MainActivity.kt).....	7
Inicialización y Gestión de Vistas.....	7
Manejo de Estados.....	8
Diseño de la Interfaz de Usuario (activity_main.xml).....	9
Estructura Base y Área de Entrada (Editor).....	9
Panel de Acciones y Retroalimentación.....	9
Motor Gráfico Nativo (VistaDiagrama.kt).....	10
Herramientas de Dibujo.....	10
Estructura de Datos del AST (FiguraDiagrama.java).....	11
Atributos de Estado (Propiedades del Nodo).....	12
Inicialización y Construcción.....	12
Diagrama de clases.....	13

Introducción.

La aplicación es un intérprete desarrollado para el sistema operativo Android utilizando el lenguaje de programación Kotlin. Para el análisis de texto se implementaron las herramientas JFlex (Analizador Léxico) y CUP (Analizador Sintáctico), las cuales generan código Java que se integra perfectamente con el entorno de Kotlin.

Organización del Proyecto y Diagrama de Clases.

El proyecto sigue el patrón de diseño vista/controlador adaptado a Android.

- **Lexer.flex / Parser.cup:** Clases generadas automáticamente por JFlex y CUP.
- **MainActivity.kt:** Actúa como el controlador principal. Captura el texto de entrada, invoca al analizador (Lexer y Parser) y maneja la visualización de los diálogos de reportes.
- **activity_main.xml:** define la estructura visual y la jerarquía de componentes.
- **VistaDiagrama.kt:** Es una clase personalizada que hereda de View. Se encarga del renderizado gráfico nativo usando Canvas, Paint y Path para dibujar figuras dinámicas basadas en el Árbol de Sintaxis Abstracta (AST).
- **FiguraDiagrama.java:** Actúa como el nodo del Árbol de Sintaxis Abstracta. Guarda el tipo de figura, el texto visible, el índice y los bloques anidados.
- **ConfiguracionVisual (Dentro de Parser):** Estructura de datos basada en HashMaps que almacena la configuración dinámica (Colores, Fuentes y Tamaños) extraída de la gramática.

Análisis de gramática: Analizador léxico (Lexer.flex).

El analizador léxico de la aplicación fue desarrollado utilizando la herramienta JFlex. Su función principal es leer la secuencia de caracteres del código fuente de entrada y agruparlos en componentes léxicos (tokens) con significado, que posteriormente serán consumidos por el analizador sintáctico (CUP).

- **Configuraciones y declaraciones generales.**

El archivo Lexer.flex está configurado dentro del paquete com.example.generadordigramas.analizador. Entre las directivas principales utilizadas se encuentran:

- **%cup:** Permite la compatibilidad e integración directa con el parser generado por Java CUP.
- **%line y %column:** Habilitan el conteo automático de líneas y columnas para la generación precisa de reportes y manejo de errores.
- **%ignorecase:** Permite que el analizador no distinga entre mayúsculas y minúsculas al leer las palabras clave, brindando flexibilidad al usuario.

Para el manejo de errores se crearon subclases estáticas ErrorLexico y un ArrayList<ErrorLexico>. Esto permite almacenar cada error detectado en una lista en lugar de detener la ejecución del programa, facilitando el reporte final.

- **Expresiones Regulares**

Para la identificación de patrones complejos, se definieron las siguientes expresiones regulares fundamentales:

- **Letras y Dígitos:** Letra = [a-zA-Z_] y Dígito = [0-9]

- Identificadores: Identificador = {Letra}({Letra}|{Digito})* (Todo identificador debe comenzar con una letra o guion bajo, seguido de cero o más letras o números).
- Números: * Entero = {Digito}+ (Uno o más dígitos consecutivos).
- Decimal = {Entero}."{Entero} (Dos grupos de enteros separados por un punto decimal).
- Cadenas de Texto: Cadena = \"[^\"]*\" (Cualquier secuencia de caracteres encerrada entre comillas dobles).
- Colores Hexadecimales: ColorHex = H[0-9a-fA-F]{6}
- Comentarios: Comentario = #[^\n]*\n? (Todo texto que inicie con # hasta el final de la línea).

- **Clasificación de tokens (Reglas léxicas).**

El analizador reconoce y clasifica los tokens en las siguientes categorías, retornando el objeto Symbol correspondiente a la clase sym de CUP:

- Palabras Reservadas del Algoritmo: Reconoce las instrucciones principales de control de flujo como INICIO, FIN, VAR, SI, ENTONCES, MIENTRAS, MOSTRAR, entre otras.
- Instrucciones de Configuración: Detecta los tokens que inician con % utilizados para el diseño del diagrama (ej. %DEFAULT, %COLOR_TEXTO_SI, %FIGURA_MIENTRAS).
- Atributos Gráficos: Detecta las palabras reservadas correspondientes a las figuras (ELIPSE, ROMBO, etc.) y fuentes tipográficas (ARIAL, COMIC_SANS, etc.).
- Operadores:
 - Aritméticos: +, -, *, /
 - Relacionales: ==, !=, <, >, <=, >=
 - Lógicos: &&, ||, !
- Signos de Agrupación y Asignación: Paréntesis (), signo de igual =, pipe | y coma ,.
- Separador de Secciones: El token especial %%%% que divide la lógica del algoritmo de su configuración visual.

- **Manejo y Recuperación de Errores Léxicos**

Para evitar que la compilación se detenga ante caracteres desconocidos, se implementaron reglas específicas para atrapar errores comunes y dar mensajes precisos:

- Cadenas sin cerrar: La expresión \"[^\"]*\n\" atrapa las cadenas a las que les falta la comilla de cierre.
- Configuraciones inválidas: La expresión \"%\"[a-zA-Z_]\"[a-zA-Z0-9_]\" atrapa comandos visuales mal escritos.

- Caracteres prohibidos: Se crearon reglas individuales para caracteres ilegales como @, \$, ^, &, ?, brindando una descripción detallada de cuál fue el símbolo no reconocido y en qué línea y columna se ubicaba.
- Regla todo: El punto (.) actúa como la regla final para atrapar cualquier otro símbolo no contemplado y reportarlo como "Token no reconocido".

Análisis de Gramática: Analizador Sintáctico (Parser.cup)

Esta es la fase principal del análisis, responsable de verificar la estructura gramatical del lenguaje, construir la representación del diagrama de flujo y recolectar las configuraciones visuales.

Código de usuario y estructuras de datos.

Para evitar la creación de múltiples clases externas y agilizar el procesamiento en memoria, se inyectó código Java directamente en el parser (parser code { : ... : }). Las estructuras de datos principales son:

- Clases de Reportes: ErrorSintactico, ReporteOperador y ReporteControl. Estas clases actúan como modelos de datos para llenar las tablas en la interfaz.
- Evaluador Matemático Integrado: Se desarrollaron métodos propios (evaluarExpresion, evaluarMultDiv, evaluarSumaResta) que evalúan expresiones aritméticas en tiempo real durante el análisis sintáctico.
- Gestor de Configuración Visual (ConfiguracionVisual): Es una clase robusta que utiliza colecciones dinámicas (HashMap<Integer, String>) para almacenar colores, formas geométricas, fuentes tipográficas y tamaños de letra asociadas al índice numérico de cada bloque (indice_opt). También define valores por defecto.

Precedencia y asociatividad de operadores.

Para resolver ambigüedades clásicas de gramáticas matemáticas y asegurar que las operaciones se evalúen en el orden correcto, se definió la precedencia (de menor a mayor):

- Lógicos: OR (izq), AND (izq), NOT (der).
- Relacionales: ==, !=, <, >, <=, >= (izq).
- Aritméticos (Suma/Resta): +, - (izq).
- Aritméticos (Multiplicación/División): *, / (izq).

Reglas de Producción.

La gramática está estructurada para dividir el archivo en dos grandes contextos analizados secuencialmente:

- Sección Principal (programa): La producción raíz exige una seccion_algoritmo, seguida estrictamente por el token SEPARADOR (%%%), y finalmente la seccion_configuracion.
- Sección de Algoritmo:
 - Inicia con INICIO y finaliza con FIN.

- Deriva en una recursividad por la izquierda de instrucciones, las cuales pueden ser simples (declaración, asignación, MOSTRAR, LEER) o de bloque (condicional, ciclo).
- Cada regla sintáctica exitosa instancia y retorna un objeto FiguraDiagrama, construyendo así secuencialmente la lista de nodos para el renderizado visual. Las estructuras de control añaden sus propias instrucciones dentro del atributo bloqueInterno del nodo.
- Captura de Ocurrencias: Cuando se reduce una operación aritmética o una instrucción condicional/ciclo, la gramática ejecuta funciones (agregarOperador, agregarControl) que extraen la línea y columna exactas para los reportes.
- Sección de Configuración: Las producciones interpretan los comandos visuales. Se destaca el manejo inteligente de los colores (expr_color), donde el parser es capaz de tomar tres expresiones aritméticas separadas por comas (formato RGB), evaluarlas matemáticamente, limitar sus valores de 0 a 255 y transformarlas automáticamente a formato hexadecimal (#RRGGBB) antes de guardarlas.

Recuperación de Errores.

El analizador es tolerante a fallos, lo que significa que un error de sintaxis no detiene la ejecución completa de la aplicación, esto se logró de dos formas:

1. Sobrescritura de Métodos Base: Se sobrescribieron `syntax_error` y `unrecovered_syntax_error` para que, en lugar de abortar, atrapen el token problemático, su ubicación y lo añadan a la lista `listaErrores`.
2. Producciones de Error Estratégicas: Se introdujo el token especial error de CUP en puntos críticos de la gramática. Ejemplos notables en el código:
 - VAR error: Captura la ausencia de un identificador en una declaración.
 - ID error: Captura la ausencia de un signo igual en una asignación.
 - SI error ENTONCES bloque_simple cierre_si: Evita que una condición mal formada destruya todo el flujo del diagrama.
 - Se evalúa si el usuario olvidó el separador %%% o los bloques de INICIO/FIN, agregando un mensaje descriptivo personalizado.

Controlador Principal ([MainActivity.kt](#))

Actúa como el controlador central de la aplicación. Es responsable de gestionar la interacción del usuario, manejar la ejecución de los analizadores léxico y sintáctico, administrar los estados de la interfaz de usuario y generar las vistas dinámicas para los reportes

Inicialización y Gestión de Vistas.

Al iniciar el ciclo de vida de la actividad, el controlador enlaza los componentes visuales definidos en el layout XML. Las vistas principales incluyen:

- Entrada de texto: Un EditText (editTextoEntrada) donde el usuario escribe el pseudocódigo.
- Acciones: Botones para iniciar el análisis (btnAnalizar) y para desplegar reportes (btnReporteErrores, btnReporteOperadores, btnReporteControl). Por defecto, los botones de reporte se inicializan deshabilitados para evitar excepciones por datos nulos.
- Salida Visual: Un ScrollView que contiene el lienzo de dibujo personalizado (VistaDiagrama) y un TextView para mensajes de estado.

Flujo de Análisis (analizarCodigo).

El método principal de ejecución sucede al presionar el botón de "Compilar". Su flujo de trabajo garantiza que el texto sea procesado de manera segura:

- Lectura de Datos: Captura el texto del EditText y lo inyecta en un StringReader.
- Ejecución del Análisis: Se instancian las clases generadas automáticamente Lexer y Parser. Inmediatamente se invoca parser.parse(), lo que desencadena la construcción del Árbol de Sintaxis Abstracta de tipo ArrayList<FiguraDiagrama>.
- Consolidación de Errores: Se extraen las listas de errores tanto del analizador léxico como del sintáctico. Ambos tipos de errores se unifican en una estructura de datos interna llamada ErrorCompleto, la cual se ordena algorítmicamente por línea y columna para presentar la información de forma cronológica al usuario.

Manejo de Estados.

Una vez finalizado el análisis, el controlador evalúa la lista unificada de errores (erroresCompleto) y separa el comportamiento de la interfaz:

- Ruta de Éxito (Sin errores): Si la lista de errores está vacía, se inyecta el árbol (lista de nodos) y la ConfiguraciónVisual extraída por el parser directamente hacia la instancia de VistaDiagrama, ordenándole dibujar el diagrama. Se habilitan los botones de Reporte de Operadores y Estructuras de Control, y se deshabilita el de Errores.
- Ruta de Fallo (Con errores): Si se detecta uno o más errores (léxicos o sintácticos), se oculta el lienzo de dibujo para evitar la renderización de un diagrama corrupto. Se muestra un mensaje en pantalla indicando la cantidad de errores encontrados y se habilita de forma exclusiva el botón de Reporte de Errores.

Generación Dinámica de Reportes.

Se implementaron métodos generadores de interfaces dinámicas (mostrarReporteErrores, mostrarReporteOperadores, mostrarReporteControl).

El motor de tablas funciona de la siguiente manera:

1. Construcción de Tabla: Crea instancias de TableLayout con la propiedad isStretchAllColumns = true para que las celdas se ajusten al ancho de la pantalla.
2. Inyección de Filas (crearFilaTabla): Un método auxiliar genera objetos TableRow. Utiliza una técnica de diseño anidado aplicando colores de fondo (BackgroundColor) y márgenes (setMargins) para simular bordes de cuadrícula perfectos entre las celdas.

Diferencia automáticamente si la fila es cabecera (texto en negrita y fondo gris) o contenido estándar.

3. **Despliegue Responsivo:** Para garantizar que tablas con datos extensos (como la de errores) no se desborden ni se recorten en pantallas móviles, el `TableLayout` resultante se envuelve simultáneamente en un `ScrollView` (scroll vertical) y un `HorizontalScrollView` (scroll horizontal) antes de ser inyectado en un componente nativo `AlertDialog`.

Diseño de la Interfaz de Usuario (activity_main.xml)

Define la estructura visual y la jerarquía de componentes (`ViewGroups` y `Views`) de la única pantalla de la aplicación. El diseño fue construido utilizando el sistema de layouts de Android, priorizando la accesibilidad, la distribución dinámica del espacio y la correcta visualización de diagramas extensos.

Estructura Base y Área de Entrada (Editor).

El contenedor raíz de la interfaz es un `LinearLayout` con orientación vertical (`android:orientation="vertical"`). Esto permite apilar los elementos de arriba hacia abajo de forma lógica:

- **Área de Edición (editTextoEntrada):** Se implementó un componente `EditText` configurado específicamente para actuar como un editor de código. Se le asignó una fuente monoespaciada (`android:fontFamily="monospace"`) para facilitar la lectura del pseudocódigo y las tabulaciones. Soporta múltiples líneas (`android:inputType="textMultiLine"`) y alinea el texto en la esquina superior izquierda (`android:gravity="top|start"`), simulando el comportamiento de un IDE estándar.

Panel de Acciones y Retroalimentación.

Debajo del editor, la interfaz presenta los controles de ejecución y estado:

- **Mensajes de Estado (txtMensaje):** Un `TextView` dinámico utilizado por el controlador para informar al usuario sobre el resultado de la compilación (ej. "Análisis exitoso" o "Se encontraron X errores").
- **Botones de Control:** Se utilizaron contenedores `LinearLayout` horizontales anidados para organizar los botones.
- El botón principal `Compilar (btnCompilar)` abarca todo el ancho disponible.
- Los botones de reportes (`Errores`, `Operadores`, `Control`) están distribuidos equitativamente en una misma fila utilizando la propiedad `android:layout_weight="1"`. Cada botón cuenta con una paleta de colores distintiva asignada mediante `backgroundTint` para guiar visualmente al usuario según la acción.

Sistema de Renderizado Bidireccional.

La parte más compleja del diseño XML es la implementación del área de dibujo del diagrama de flujo. Dado que los algoritmos pueden generar diagramas muy altos o muy anchos, el lienzo debía soportar desplazamiento en ambas direcciones. Esto se resolvió mediante la siguiente jerarquía anidada:

1. **ScrollView (scrollDiagrama):** Actúa como el contenedor principal asignándole el resto del espacio disponible en la pantalla (`android:layout_weight="1"`). Permite el desplazamiento vertical.
2. **HorizontalScrollView:** Anidado directamente dentro del **ScrollView** vertical, habilitando el desplazamiento de izquierda a derecha.
3. **FrameLayout:** Actúa como contenedor puente para la vista personalizada.
4. **Vista Personalizada (VistaDiagrama):** El componente final es la etiqueta XML que invoca la clase nativa de Kotlin `com.example.generadordiagramas.VistaDiagrama`.
 - **Virtualización del Espacio:** Se le asignaron dimensiones fijas extremadamente amplias (`android:layout_width="1200dp"` y `android:layout_height="1500dp"`). Esto crea un "pizarrón virtual" gigante sobre el cual el método `onDraw` de Kotlin puede trazar las ramificaciones de las sentencias SI y ciclos MIENTRAS sin que el sistema operativo corte o limite los bordes del dibujo.

Motor Gráfico Nativo (VistaDiagrama.kt)

Es un componente personalizado que hereda de la clase base `View` de Android. Su responsabilidad exclusiva es el renderizado en 2D del diagrama de flujo. Actúa como el puente visual que transforma el **Árbol de Sintaxis Abstracta** de las figuras lógicas y el objeto de `ConfiguracionVisual` en píxeles sobre la pantalla.

Herramientas de Dibujo.

Para garantizar un trazo nítido y eficiente, la vista utiliza la API gráfica nativa de Android a través de la clase `Canvas`. Se inicializan estáticamente cuatro objetos `Paint` (pinceles) con la propiedad `isAntiAlias = true` para suavizar los bordes:

- `pincelRelleno`: Utilizado para pintar el color de fondo dinámico de las formas (`Style.FILL`).
- `pincelBorde`: Traza el contorno perimetral negro de las figuras geométricas (`Style.STROKE`).
- `pincelTexto`: Responsable de la tipografía, configurado para centrar el texto automáticamente (`Align.CENTER`).
- `pincelLinea`: Utilizado para dibujar las flechas o líneas de flujo que conectan los bloques lógicos.

Intérpretes de Configuración Dinámica

Dado que el analizador léxico extrae la configuración en formato de cadenas de texto, la clase implementa métodos traductores seguros:

- `interpretarColor`: Recibe una cadena (por ejemplo, `HFF0000` o `#FF0000`) y la parsea a un entero de color de Android mediante `Color.parseColor`. Incorpora un bloque `try-catch` que actúa como fallback, garantizando que si el color es inválido, se aplique el color por defecto en lugar de colapsar la aplicación.

- obtenerFuente: Mapea los identificadores tipográficos de la gramática (ARIAL, COMIC_SANS, TIMES_NEW_ROMAN, VERDANA) a la clase Typeface de Android. Para lograr fuentes específicas como Comic Sans, se utiliza el generador Typeface.create("casual", ...) del sistema operativo.

Generación Geométrica (dibujarFormaDinamica).

Este método centraliza la lógica matemática para trazar los perímetros de las figuras.

Dependiendo de la variable figura, el motor decide cómo actuar:

- Formas simples como ELIPSE, CIRCULO y RECTANGULO utilizan métodos directos del Canvas (drawOval, drawRect, drawRoundRect con soporte para RectF).
- Formas poligonales complejas como el ROMBO y el PARALELOGRAMO se construyen vectorialmente mediante la clase Path. El algoritmo traza vértices (moveTo, lineTo) calculando desplazamientos geométricos basados en las coordenadas izquierda, arriba, derecha y abajo, aplicando una inclinación matemática para crear el efecto de paralelogramo.

Algoritmo de Renderizado y Recursividad (dibujarBloque).

El trazado principal ocurre en el ciclo dibujarBloque, el cual se ejecuta en el método onDraw.

Su lógica de diseño estructural opera de la siguiente manera:

1. Recorrido Lineal: Itera sobre la lista de nodos, por cada nodo, calcula sus coordenadas espaciales (X, Y) y aumenta un margen de separación vertical (espacioVertical) para preparar el lugar del siguiente nodo.
2. Inyección de Estilos: Se consulta el objeto ConfiguraciónVisual (cv) utilizando el índiceElemento del nodo actual. Se extrae el color de fondo, color de texto, tipo de fuente, tamaño de letra y tipo de figura (diferenciando entre bloques de PROCESO, SI o MIENTRAS). Si no existe configuración para ese índice, se hereda la configuración global/por defecto.
3. Manejo de Ramificaciones (Recursividad): Si un nodo de tipo CONDICION posee instrucciones anidadas (bloqueInterno no nulo ni vacío), la vista dibuja una línea horizontal indicando el camino verdadero ("V"). Inmediatamente, invoca a sí misma (dibujarBloque) pasándole la sub-lista de nodos y un desplazamiento hacia la derecha (x + 600f).
4. Cierre de Flujo: Finalmente, la función retorna la coordenada "Y" máxima alcanzada, asegurando que si una rama secundaria crece más que la principal, el siguiente bloque no se sobreponga y el diagrama mantenga una estructura limpia y ordenada de arriba hacia abajo.

Estructura de Datos del AST (FiguraDiagrama.java)

Funciona como un "Data Transfer Object" (DTO) y representa un nodo individual dentro del Árbol de Sintaxis Abstracta (AST) generado por el analizador sintáctico.

Esta clase es el puente de comunicación entre la capa de procesamiento lógico (Parser) y la capa de presentación visual (VistaDiagrama), almacenando toda la información semántica y estructural de cada instrucción del pseudocódigo.

Atributos de Estado (Propiedades del Nodo).

La clase encapsula la información del bloque lógico a través de cuatro atributos públicos principales:

- tipoForma (String): Define la categoría semántica de la instrucción analizada (ej. "INICIO", "FIN", "PROCESO", "IO", "CONDICION"). El motor gráfico utiliza este valor para determinar qué figura geométrica base debe dibujar.
- textoVisible (String): Almacena el contenido literal que será renderizado dentro de la figura geométrica (ej. "VAR a = 10", "a < b" o "Fin del programa").
- bloqueInterno (ArrayList<FiguraDiagrama>): Es el atributo más complejo, ya que le otorga la característica de recursividad a la estructura de datos. Cuando el parser detecta una estructura de control (como un SI o un MIENTRAS), todas las instrucciones que pertenecen a ese bloque se anidan dentro de esta lista. Si el nodo es una instrucción simple (como un PROCESO), este atributo permanece como nulo.
- indiceElemento (int): Actúa como un identificador único numérico para el nodo. Es crucial para el sistema de configuración dinámica, ya que permite enlazar una instrucción específica con un diseño visual personalizado (por ejemplo, relacionar el segundo ciclo MIENTRAS con el comando %FIGURA_MIENTRAS=CIRCULO|2).

Inicialización y Construcción.

El constructor de la clase public FiguraDiagrama(String tipoForma, String textoVisible) está diseñado para instanciar rápidamente nodos base.

Al momento de su creación, exige los dos parámetros obligatorios (el tipo semántico y el texto a mostrar) e inicializa por defecto el bloqueInterno como null y el indiceElemento en 0 (indicando que, salvo que el parser asigne un índice explícito posteriormente, el nodo heredará la configuración visual global o por defecto).

Diagrama de clases

